

Preston-Check Whitepaper

Preston-Check: Enterprise Security Audit Suite for Financial Platforms

A Whitepaper by Preston Security — April 2026

Executive Summary

Financial platforms face a unique security challenge. Generic security tools scan for common vulnerabilities — SQL injection, XSS, outdated dependencies — but miss the attack patterns specific to money movement: structuring transactions to avoid regulatory thresholds, exploiting race conditions in balance calculations, manipulating exchange rates through stale price feeds, or using dormant accounts as money laundering conduits.

Preston-Check was built to close this gap. Forged in the aftermath of a sophisticated, multi-vector attack against a live financial platform, the tool encodes 100 check categories with 276 individual test points, covering not only the standard security surface but the financial-specific behavioral, mathematical, and regulatory patterns that no other tool addresses. It maps to six compliance frameworks — PCI-DSS v4.0, SOC 2 Type II, ISO 27001:2022, OWASP API Top 10, NIST CSF 2.0, and CIS Controls v8 — with 100% coverage across all six.

Preston-Check is more than an audit tool. It is the foundation of a Continuous Defense model that runs detection rules against production behavior every 60 seconds, applies graduated responses with fail-safe guardrails, and feeds findings into a self-improving Virtuous Cycle that ensures the security posture of the platform improves with every iteration.

The Problem

Generic tools miss financial risk

Traditional security scanners like SonarQube, Snyk, and OWASP ZAP are designed for general-purpose web applications. They catch injection vulnerabilities, dependency CVEs, and misconfigured headers. They do not catch that a payment endpoint uses `double` instead of `BigDecimal` for monetary arithmetic (introducing IEEE 754 floating-point errors that accumulate into reconciliation discrepancies), that a withdrawal handler lacks a cooling period for new destination addresses (enabling instant theft after account

takeover), or that three deposits just under \$10,000 in a single day trigger the Bank Secrecy Act's structuring threshold (a federal crime that the platform is obligated to detect and report).

Financial platforms need security tools that understand money.

Point-in-time audits leave gaps

Annual penetration tests and quarterly vulnerability scans produce snapshots. Between snapshots, new code is deployed, new endpoints are exposed, and new attack patterns emerge. The gap between the last audit and the next deployment is when attackers strike.

Preston-Check runs continuously. The light mode completes in 30 seconds and integrates into pre-deployment gates. The full mode runs 100 checks in under 4 minutes. The Continuous Defense engine runs every 60 seconds in production, watching for behavioral anomalies that no static scan can detect.

Compliance is fragmented

A fintech platform must comply with multiple overlapping frameworks. PCI-DSS governs card data. SOC 2 governs service organization controls. ISO 27001 provides a comprehensive management framework. OWASP covers API security. NIST provides the cybersecurity framework. CIS provides implementation guidance. Each framework has its own assessment methodology, its own documentation requirements, and its own audit cadence.

Preston-Check unifies these into a single run. Every check is mapped to every applicable framework requirement. One tool, one report, six frameworks satisfied.

The Solution

100 Check Categories in 14 Themes

Preston-Check organizes its 100 checks into 14 themes that progress from code-level scanning through infrastructure, compliance, and behavioral analysis to mathematical verification.

Themes 1 through 3 (P-01 to P-20) cover code scanning, live monitoring, and platform security. These are the foundational checks that every security tool should have: hardcoded secrets, authentication bypass, injection, session management, encryption, error handling, and deployment safety. Preston-Check's versions of these checks are informed by real attack forensics rather than theoretical vulnerability databases, making them more precise and less prone to false positives.

Themes 4 through 9 (P-21 to P-49) cover compliance, infrastructure, API security, data protection, operational security, supply chain, and mobile/frontend. These checks address the OWASP API Top 10 with dedicated tests for each risk, infrastructure hardening for cloud-native deployments, and supply chain security including SBOM generation and dependency pinning.

Theme 10 (P-50 to P-56) is the original Finance-Specific Gold Standard. These checks verify that the fundamental patterns of a financial system are implemented correctly: BigDecimal for monetary arithmetic, privilege escalation protection, timing attack prevention, account lifecycle management, API key governance, cryptocurrency address validation, and multi-signature custody approval workflows.

Theme 11 (P-57 to P-64) fills compliance gaps that traditional tools ignore: business logic abuse (OWASP API6), vulnerability scan evidence, security policy documentation, email authentication, privacy and consent, penetration test evidence, supply chain risk management, and recovery testing.

Theme 12 (P-65 to P-82) is the extended Finance-Specific suite that differentiates Preston-Check from every other tool on the market. Transaction velocity and structuring detection. Dormant account reactivation monitoring. Cross-account transfer and money mule detection. Fee manipulation prevention. Exchange rate safety with stale rate protection. Settlement finality and immutability. Beneficial ownership tracking. Sanctions and PEP screening. Transaction limit enforcement with atomic database operations. Proof of reserves and balance reconciliation. Financial audit trail immutability. Payment state machine validation. Withdrawal controls with address whitelisting and cooling periods. Ledger consistency and zero-sum validation. Regulatory reporting readiness. Financial event sourcing. Input validation guards against overflow, negative amounts, NaN, and type coercion. Continuous defense model readiness.

Theme 13 (P-83 to P-95) is the compliance evidence verification layer. These checks search for the organizational artifacts that auditors require — policies, training records, risk assessments, vendor reviews, physical security documentation. They search codebase patterns first (badge system integration code, training platform API references, CloudWatch configurations) and fall back to document existence checks. This is how Preston-Check achieves 100% compliance framework coverage: by verifying evidence of the organizational controls that pure code scanners miss.

Theme 14 (P-96 to P-100) is the mathematical invariant verification suite. These five checks verify the properties that, if broken, mean the financial system is fundamentally unsound. Integer overflow protection (Java's long wraps silently at 9.2 quintillion). Division precision with consistent rounding modes (inconsistent rounding creates penny discrepancies that accumulate). Currency safety preventing cross-currency arithmetic (adding USD to EUR is a category error). Boundary value protection (exactly \$10,000 must trigger CTR — is the check $\geq$ or $>$?). And the five mathematical invariants that every financial system must preserve: conservation of value, non-negativity of balances, commutativity of reversals, idempotency of completed operations, and monotonicity of transaction identifiers.

The Virtuous Cycle

Preston-Check is not a point-in-time tool. It is the engine of a Virtuous Cycle — a structured, repeatable process that runs the audit, compares results to the previous cycle, learns through three analysis loops (pattern recognition, root cause analysis, and preventive measures), proposes specific remediations, and waits for human approval before applying any changes. The score can only go up, never regress.

The cycle operates through six phases: Diagnose (run the audit), Compare (compute delta from previous cycle), Learn (three analysis loops), Propose (generate remediation items), Await (human approval in the Operations Portal), and Apply (implement, build, deploy, verify). Each cycle builds on the last. The improvements compound over time.

Continuous Defense

Beyond the audit cycle, Preston-Check powers a Continuous Defense engine that runs in production. Eight detection rules monitor behavioral patterns in real time: transaction velocity spikes, impossible travel (IP geolocation jumps), structuring (deposits just under CTR thresholds), dormant account reactivation, rapid deposit-then-withdraw layering, new destination large withdrawals, brute force authentication, and session anomalies.

When a rule fires, the engine applies a graduated response through a six-level escalation ladder: LOG, THROTTLE, CHALLENGE, HOLD, FREEZE, ALERT HUMAN. Every response is fail-safe — the system can hold, block, and pause, but never release, approve, or move money. Four independent guardrails prevent overreaction: per-rule maximum automation caps, cooldown timers, configurable confidence thresholds, and mandatory human decision-making for irreversible actions.

Self-Healing with Guardrails

The Continuous Defense engine is part of a broader self-healing architecture that includes a session sentinel (cleaning up zombie Redis sessions every 10 seconds), a payment expiration cron (releasing orphaned HOLD transactions), health endpoint monitoring for automatic failover, and the detection/response engine itself.

Self-healing follows three inviolable laws. First, every automated action must be fail-safe: the worst outcome of a false positive must be less harmful than the worst outcome of a missed attack. Second, the escalation ladder must be followed sequentially: no jumping from LOG to FREEZE. Third, every action must be logged to an immutable audit trail.

Compliance Coverage

Preston-Check achieves 100% coverage across six major compliance frameworks.

PCI-DSS v4.0: All 12 requirements are addressed, from network security (Req 1-2) through account data protection (Req 3-4), vulnerability management (Req 5-6), access control (Req 7-8), physical access (Req 9 via P-83), monitoring (Req 10-11), and organizational policies (Req 12 via P-84).

SOC 2 Type II: All five Trust Services Criteria are covered. Security through CC6/CC7 checks. Availability through capacity planning and recovery testing (P-85). Processing Integrity through financial precision checks (P-50, P-96-P-100). Confidentiality through data classification and DLP (P-86). Privacy through consent management and DSAR handling (P-61).

ISO 27001:2022: All 93 Annex A controls across four themes. Organizational (A.5.x via P-87), People (A.6.x via P-88), Physical (A.7.x via P-89), and Technological (A.8.x via P-01 through P-82).

OWASP API Security Top 10 (2023): All 10 risks have dedicated checks, including API6 (Unrestricted Access to Sensitive Business Flows) which is frequently missed by other tools.

NIST CSF 2.0: All six functions covered. Govern (P-90), Identify (P-91), Protect (P-01 through P-56), Detect (P-10, P-38, P-39, P-82), Respond (P-40), and Recover (P-92).

CIS Controls v8: All 18 control groups addressed, from enterprise asset inventory (P-93) through penetration testing (P-95).

Technical Architecture

Preston-Check is implemented as a collection of 100 bash scripts in a `checks/` directory, orchestrated by a main runner script. Each check follows a consistent pattern: set up the source directory, run detection logic using `grep/find` against the codebase, and call a `record` function with the result (PASS, FAIL, WARN, or SKIP).

The runner supports three modes. Light mode (`--light`) runs P-01 through P-20 in approximately 30 seconds, suitable for pre-commit or pre-deploy gates. Full mode (`--full`) runs all 100 checks in under 4 minutes. CI mode (`--ci`) exits with code 1 on any FAIL, integrating with GitHub Actions, GitLab CI, or any pipeline that uses exit codes.

Results are written to both local files (timestamped cycle directories) and a PostgreSQL database (`security_audit_cycle` and `security_remediation_item` tables) for display in the Operations Portal.

The Continuous Defense engine is implemented as a Java service (`ContinuousDefenseService.java`) that uses raw JDBC to query existing production tables. It requires no new data collection infrastructure — it rides on top of the audit framework that is already capturing every API request.

Configuration is per-project via YAML files. A compliance evidence template directory provides the organizational document structure needed for 100% framework coverage.

Origin Story

Preston-Check was born from a real incident, not a theoretical exercise. A determined adversary targeted a live financial platform using a multi-vector approach that combined social engineering, technical exploitation, and regulatory threshold awareness. The attacker created synthetic identities, disabled security controls through configuration gaps, automated reconnaissance at scale, extracted sensitive internal data from API responses, and probed cryptocurrency withdrawal infrastructure for exploitable weaknesses.

The forensic investigation that followed analyzed millions of log entries and revealed a pattern: every vulnerability the attacker exploited was one that generic security tools would have missed. The 2FA bypass was not a code flaw but a policy gap. The information leakage was not a missing header but a business logic exposure. The withdrawal probing was not a brute force attack but a sophisticated understanding of custodial infrastructure.

Preston-Check encodes every lesson from that investigation — and from every engagement since — into automated, repeatable checks. The tool is named as a permanent reminder that security is ultimately tested not by scanners but by adversaries. The best defense is one that thinks like an attacker and moves faster.

Deployment Options

Preston-Check operates in three deployment models. As a global tool installed at `~/DEV/preston-check/`, it can scan any project on the developer's machine with a project-specific config file. As an embedded tool copied into a project's `tools/` directory, it integrates with that project's CI/CD pipeline. As a portal-integrated tool connected to the Operations Portal, it provides a browser-based workflow for reviewing findings, approving remediations, and tracking score progression over time.

Conclusion

Financial platforms cannot be secured by generic tools designed for general-purpose web applications. The attack surface of a money-movement system includes mathematical precision, regulatory thresholds, behavioral patterns, settlement finality, and audit immutability — dimensions that no standard security scanner addresses.

Preston-Check fills this gap with 100 check categories covering code, infrastructure, compliance, financial behavior, organizational evidence, and mathematical invariants. It maps to six compliance frameworks with 100% coverage. It powers a Continuous Defense model with fail-safe graduated response. And it drives a Virtuous Cycle that ensures the security posture improves with every iteration.

The tool carries its name as a reminder that real adversaries are the ultimate test of any security system. Preston-Check ensures that every lesson learned from every incident is encoded, automated, and enforced — not just once, but continuously.

Preston-Check Enterprise Security Suite v4.0 100 Check Categories | 276 Test Points | 6 Compliance Frameworks | 100% Coverage Preston Security, 2026. All rights reserved.
Contact: info@preston-check.com